Projet BTS SIO SISR — NAS et services auto-hébergés

- **1. Contexte et objectifs**
- Maquette virtuelle (VMware) d'un NAS + services auto-hébergés, en vue d'un futur déploiement sur portable physique. Objectifs : stockage réseau, sauvegarde, conteneurisation.
- Ce projet consiste à mettre en place un NAS (serveur de stockage en réseau) auto-hébergé, afin de disposer de mon propre espace de stockage et de plusieurs services personnels : cloud, médiathèque, filtrage DNS, VPN et hébergement de mon portfolio. Il me permet d'apprendre l'administration des systèmes et des réseaux et de constituer un projet complet pour mon BTS SIO SISR. La maquette est d'abord réalisée dans une machine virtuelle VMware, avant un futur déploiement sur un ordinateur portable dédié.
- **2. Environnement**
- VMware Workstation · OpenMediaVault (Debian) · VM 4 Go RAM (modifier pour 6 Go de ram vers la fin du projet) · 3 disques *thin* (système 20 Go, données 10 Go, sauvegarde 10 Go).
- Identifiants : console = root ; interface web OMV = admin.
- La machine virtuelle est configurée avec 4 Go de mémoire (portée ensuite à 6 Go), 4 cœurs, une carte réseau en NAT et trois disques distincts : un disque système de 20 Go, un disque de données de 10 Go et un disque de sauvegarde de 10 Go. Les disques sont séparés pour isoler le système, les données et leurs sauvegardes. Ils sont créés en « thin provisioning » (allocation progressive) afin d'économiser l'espace du disque hôte, qui était limité.

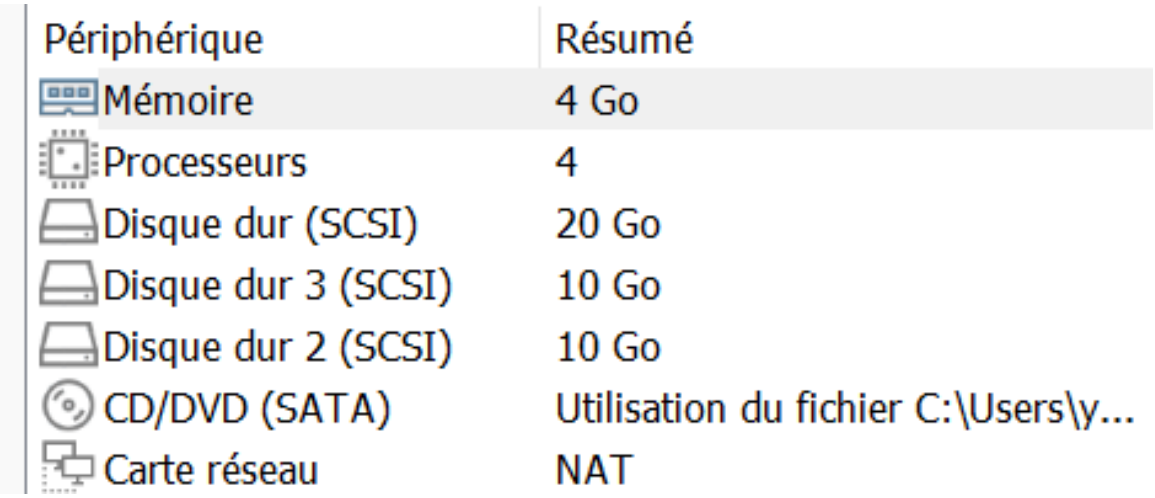
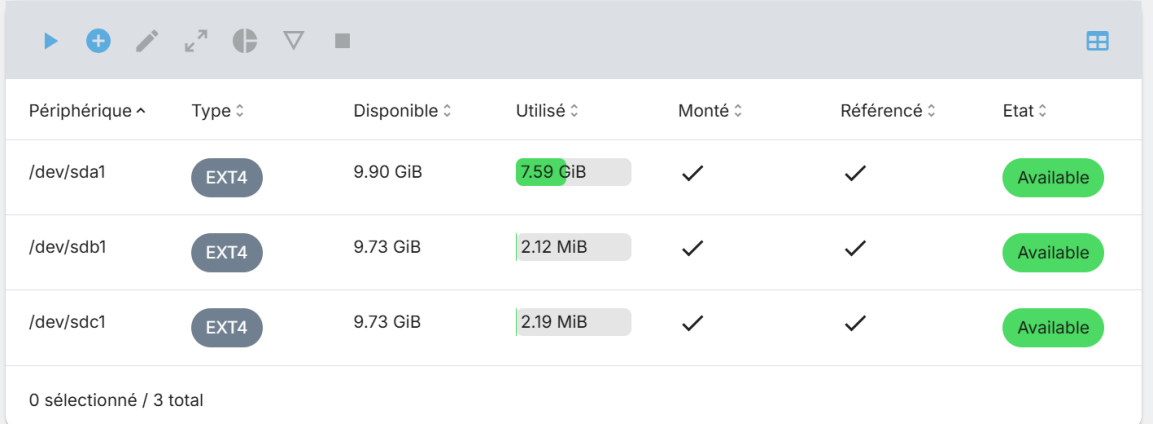


Figure 1 - Parametres de la VM : 4 Go de RAM, 3 disques (20/10/10 Go), reseau NAT

- **3. Installation et partages**

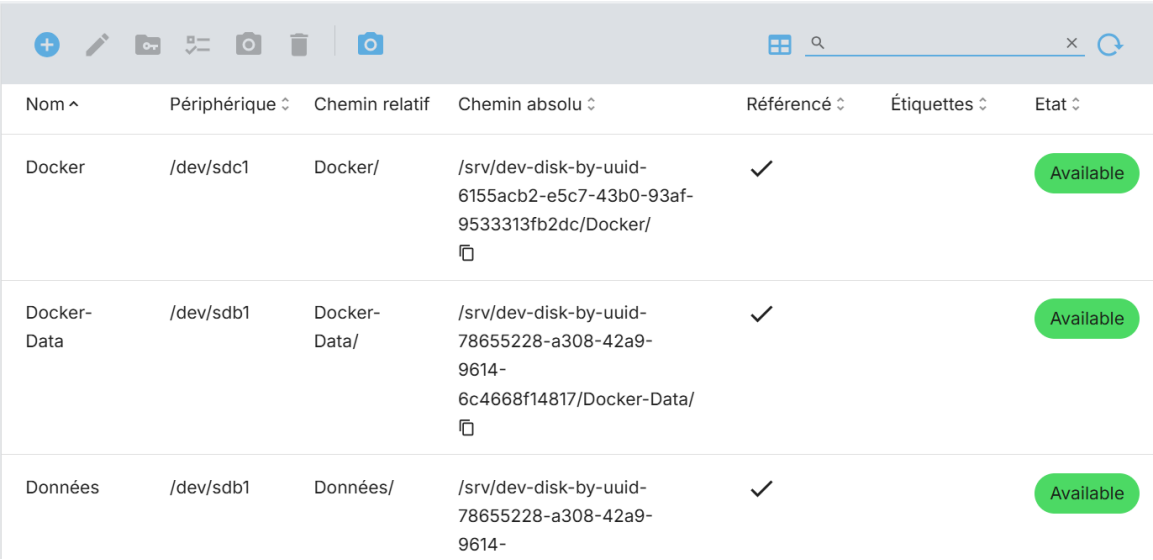
- Préparation disques (wipe → EXT4 → montage), dossier partagé, utilisateur, droits, activation **SMB/CIFS**. Accès Windows via \\192.168.108.128\Donnees.
- Après l'installation d'OpenMediaVault, j'ai préparé les disques : effacement (wipe), puis formatage en EXT4 et montage. J'ai ensuite créé un dossier partagé « Données », un utilisateur et ses droits, puis activé le service de partage SMB/CIFS. Le partage est accessible depuis Windows en tapant l'adresse du NAS dans l'explorateur de fichiers, comme un lecteur réseau.



Périphérique	Type	Disponible	Utilisé	Monté	Référencé	Etat
/dev/sda1	EXT4	9.90 GiB	7.59 GiB	✓	✓	Available
/dev/sdb1	EXT4	9.73 GiB	2.12 MiB	✓	✓	Available
/dev/sdc1	EXT4	9.73 GiB	2.19 MiB	✓	✓	Available

0 sélectionné / 3 total

Figure 2 - Systemes de fichiers : les 3 disques EXT4 montes



Nom	Périphérique	Chemin relatif	Chemin absolu	Référencé	Étiquettes	Etat
Docker	/dev/sdc1	Docker/	/srv/dev-disk-by-uuid-6155acb2-e5c7-43b0-93af-9533313fb2dc/Docker/	✓		Available
Docker-Data	/dev/sdb1	Docker-Data/	/srv/dev-disk-by-uuid-78655228-a308-42a9-9614-6c4668f14817/Docker-Data/	✓		Available
Données	/dev/sdb1	Données/	/srv/dev-disk-by-uuid-78655228-a308-42a9-9614-	✓		Available

Figure 3 - Dossiers partages (1/2)

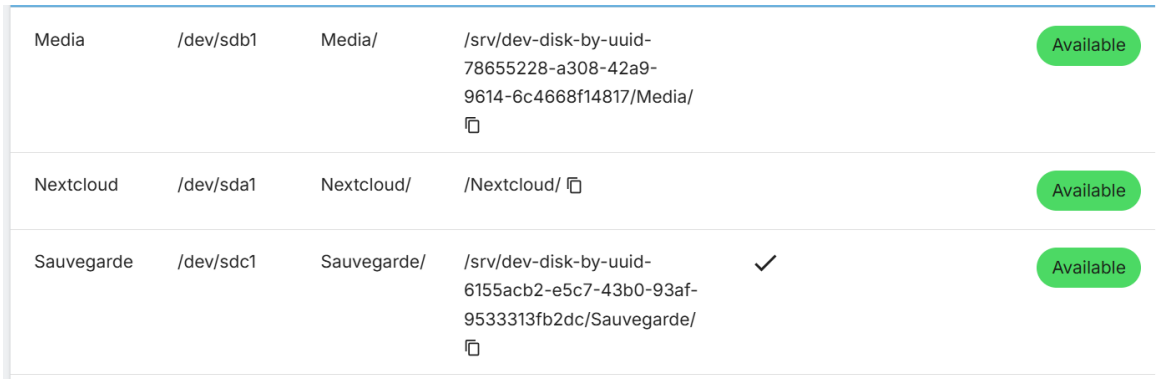


Figure 4 - Dossiers partages (2/2)

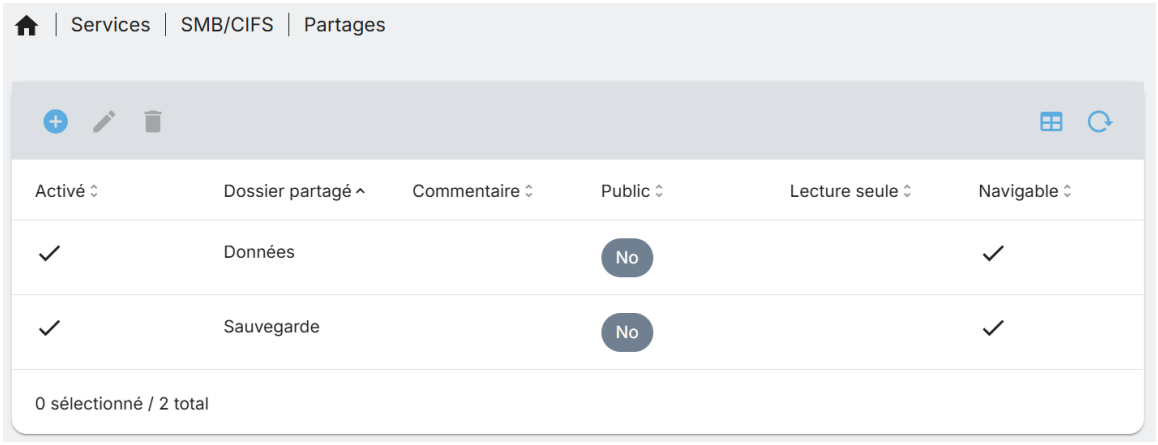


Figure 5 - Partage SMB/CIFS active (Donnees, Sauvegarde)

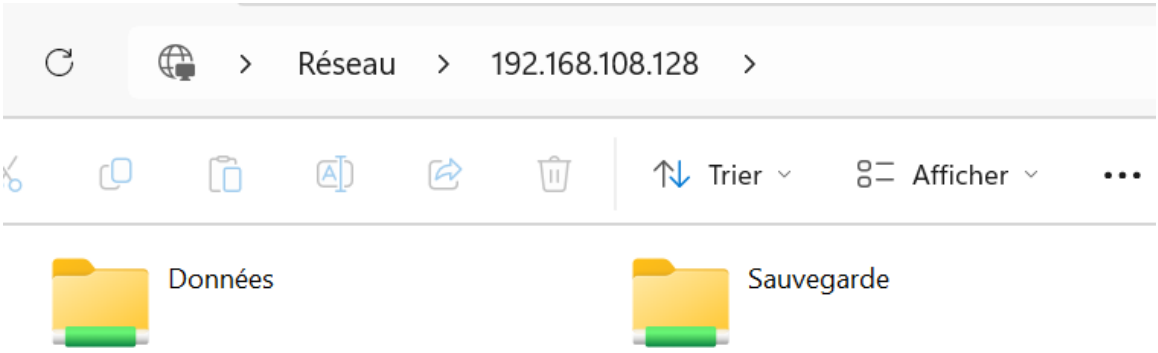


Figure 6 - Acces au partage depuis l'explorateur Windows

- **4. Stratégie de sauvegarde**
- Tâche **rsync** Données → Sauvegarde, quotidienne, option "Supprimer" désactivée. Distinction **RAID ≠ sauvegarde**. **Test de restauration** : suppression volontaire → restauration → vérification.
- J'ai mis en place une sauvegarde automatique avec rsync, qui copie chaque nuit (à 01h00) le contenu du dossier « Données » vers le dossier « Sauvegarde », sans

suppression afin de conserver les fichiers effacés par erreur. J'ai ensuite testé la restauration : suppression volontaire de fichiers, restauration depuis la sauvegarde, puis vérification que les fichiers étaient bien récupérés et intacts. Le journal de rsync (« The synchronisation has completed successfully ») confirme le bon déroulement de la sauvegarde et de la restauration. Il faut aussi retenir que le RAID n'est pas une sauvegarde : il protège d'une panne disque, pas d'une suppression accidentelle.

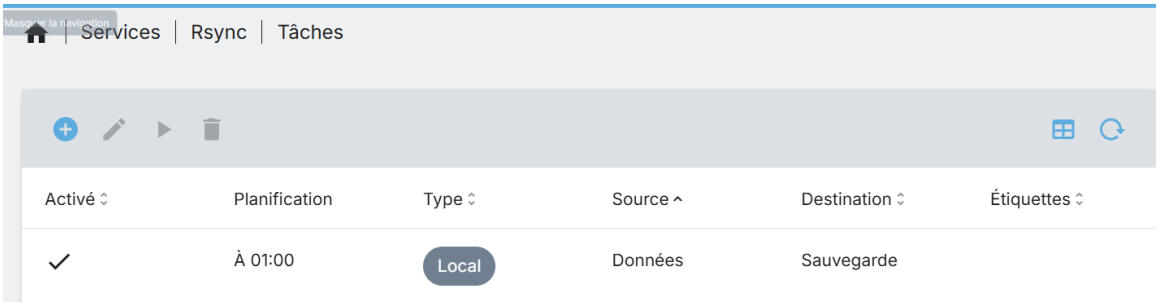


Figure 7 - Tache rsync : Donnees -> Sauvegarde, planifiee a 01:00

Lancer la tâche rsync

```
Please wait, syncing </srv/dev-disk-by-uuid-78655228-a308-42a9-9614-6c4668f14817/Données/> to </srv/dev-
disk-by-uuid-6155acb2-e5c7-43b0-93af-9533313fb2dc/Sauvegarde/> ...

sending incremental file list
Voc/

sent 288 bytes  received 16 bytes  608.00 bytes/sec
total size is 56,807  speedup is 186.87
The synchronisation has completed successfully.

END OF LINE
```

Figure 8 - Log de sauvegarde : « completed successfully »

Lancer la tâche rsync

```
Please wait, syncing </srv/dev-disk-by-uuid-6155acb2-e5c7-43b0-93af-9533313fb2dc/Sauvegarde/> to </srv/dev-
disk-by-uuid-78655228-a308-42a9-9614-6c4668f14817/Données/> ...

sending incremental file list

sent 291 bytes  received 13 bytes  608.00 bytes/sec
total size is 56,807  speedup is 186.87
The synchronisation has completed successfully.

END OF LINE
```

Figure 9 - Log de restauration : Sauvegarde -> Donnees

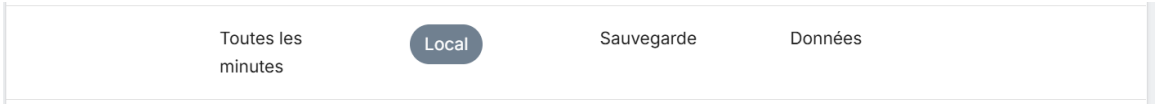


Figure 10 - Detail de la tache (type Local, source, destination)

- **5. Configuration réseau (IP statique)**

- Attribution d'une IP statique (adresse, masque, passerelle, DNS). Validation par tests de connectivité :
- ping -c 4 8.8.8.8
- ping -c 4 deb.debian.org
- J'ai attribué une adresse IP statique au NAS (192.168.108.128), avec son masque (255.255.255.0), sa passerelle (192.168.108.2) et un serveur DNS (8.8.8.8). Une adresse fixe est indispensable pour un serveur, qui doit toujours être joignable à la même adresse. J'ai validé la configuration par des tests de connectivité : un ping vers une adresse IP (passerelle fonctionnelle) et un ping vers un nom de domaine (résolution DNS fonctionnelle).

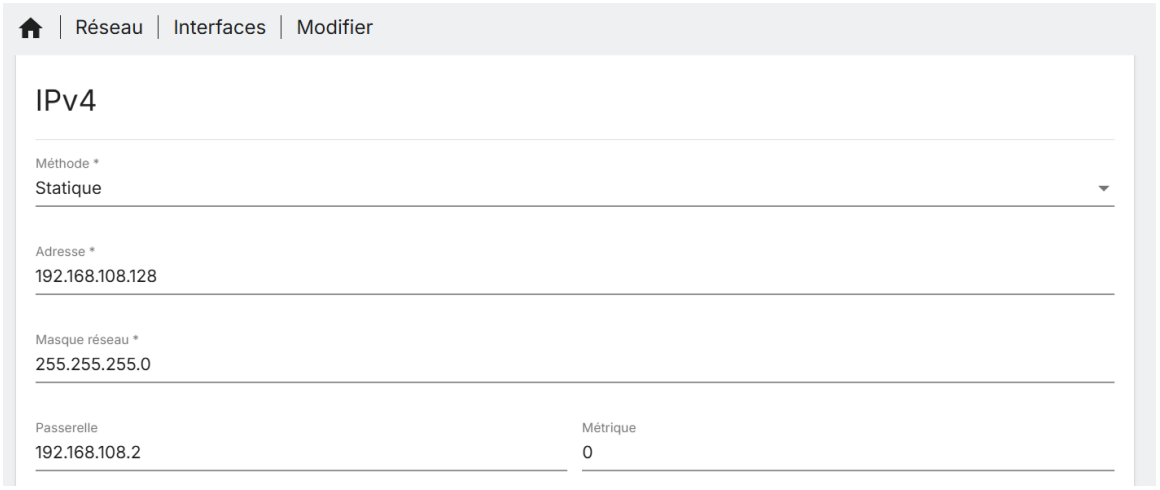


Figure 11 - Configuration reseau : IP statique, masque, passerelle



Figure 12 - Serveur DNS (8.8.8.8)

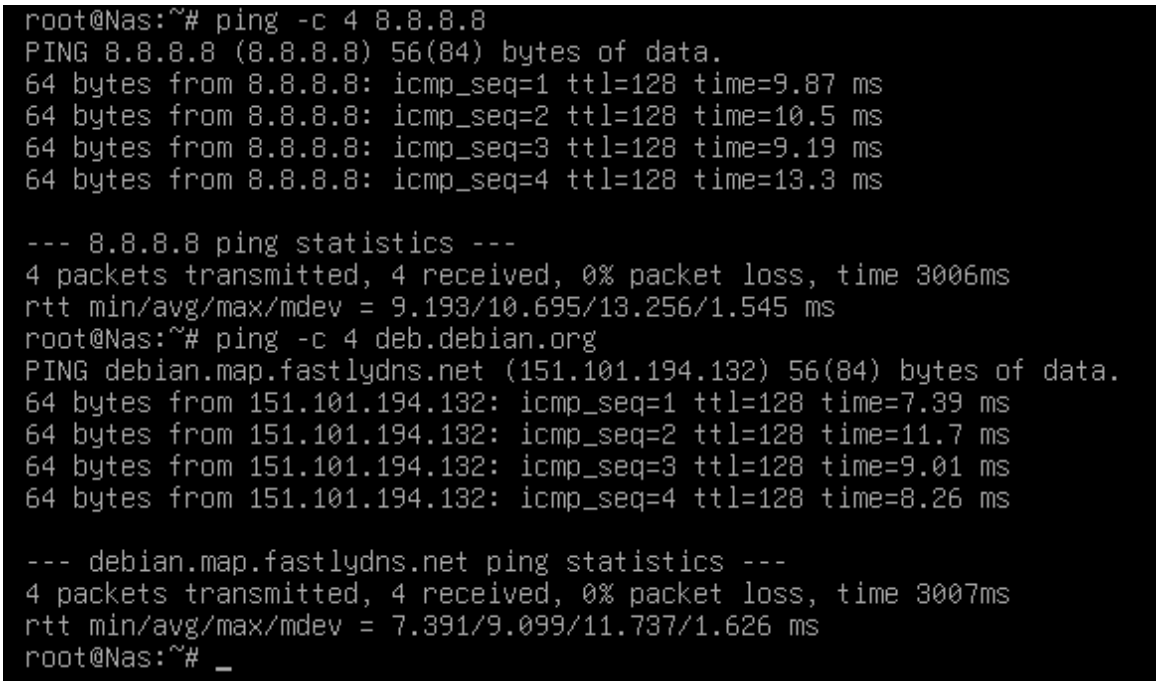


Figure 13 - Tests de connectivite : ping IP + resolution de nom

- **6. Conteneurisation avec Docker**
- Concept : image / conteneur / volume / Compose (isolation, reproductibilité). Installation :
- wget -O - https://github.com/OpenMediaVault-Plugin-Developers/packages/raw/master/install | bash
- Puis activation du dépôt Docker (omv-extras) + plugin **openmediavault-compose**.
- Pour héberger mes services, j'ai installé Docker via le plugin omv-extras, puis le plugin Compose. La conteneurisation consiste à déployer chaque application dans un conteneur isolé, avec toutes ses dépendances, sans conflit et de façon facilement reproductible (comme des conteneurs maritimes standardisés que l'on empile). Chaque service est décrit dans un fichier « compose » puis démarré (Up).

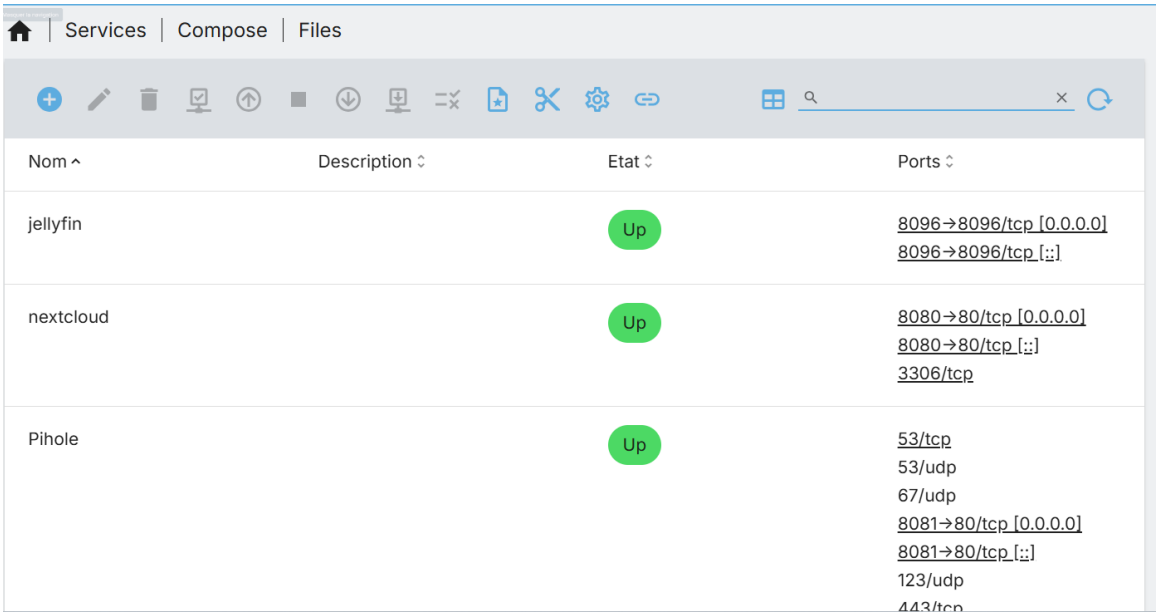


Figure 14 - Menu Compose : les stacks en etat « Up »

- **7. Services déployés**
- **Nextcloud** → :8080 (cloud perso)
- **Pi-hole** → :8081/admin (filtrage DNS)
- **Jellyfin** → :8096 (médiathèque)
- J'ai déployé trois services conteneurisés : Nextcloud (port 8080), un cloud personnel pour stocker et synchroniser mes fichiers ; Pi-hole (port 8081), un service de filtrage DNS qui bloque les publicités à l'échelle du réseau ; et Jellyfin (port 8096), une médiathèque pour films et musique. La commande « docker ps » affiche l'ensemble des conteneurs en cours d'exécution.

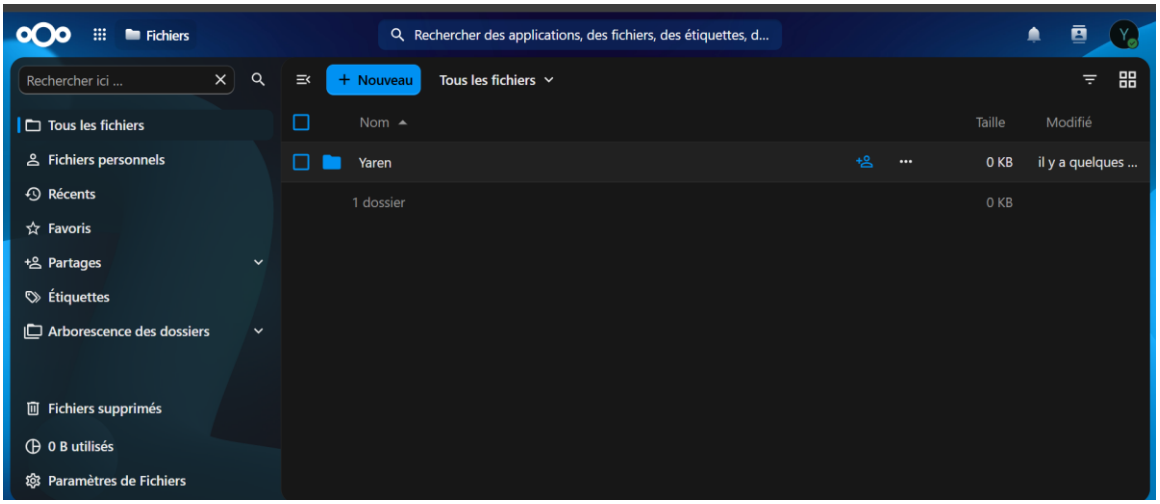


Figure 15 - Nextcloud (cloud personnel)

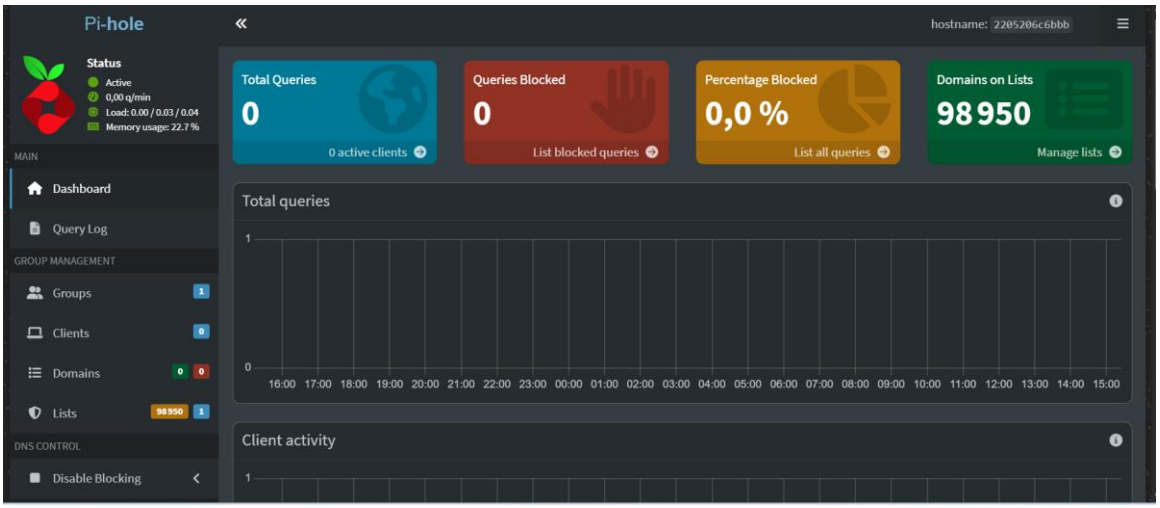


Figure 16 - Pi-hole (tableau de bord de filtrage DNS)

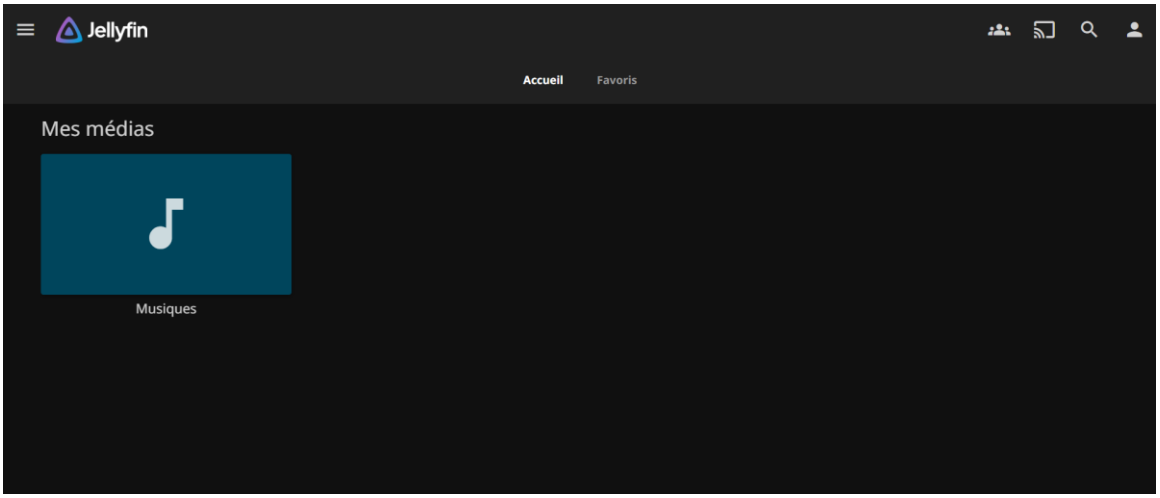


Figure 17 - Jellyfin (mediatheque)

```
root@Nas:~# docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED       STATUS                    PORTS
                                NAMES
01356771a1a6   jellyfin/jellyfin:latest       "/jellyfin/jellyfin"     3 days ago    Up 29 minutes (healthy)   0.0.0.0:8096->8096/tcp, [::]:8096->8096/tcp
                                jellyfin
2205206c6bbb   pihole/pihole:latest           "start.sh"               3 days ago    Up 29 minutes (healthy)   53/udp, 67/udp, 53/tcp, 443/tcp, 123/udp, 0.0.0.0:8081
->80/tcp, [::]:8081->80/tcp   pihole
08f431c57b08   nextcloud:latest               "/entrypoint.sh apac…"   3 days ago    Up 29 minutes             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
                                nextcloud-app
aa233bacdfed   mariadb:10.11                  "docker-entrypoint.s…"   3 days ago    Up 29 minutes             3306/tcp
                                nextcloud-db
root@Nas:~# _
```

Figure 18 - docker ps : les conteneurs en cours d'execution

8 . Accès distant sécurisé — VPN WireGuard

WireGuard déployé via wg-easy (conteneur Docker). Interface web de gestion des clients sur le port 51821 ; tunnel VPN sur le port 51820/UDP. L'accès à l'interface est protégé par un mot de passe haché (bcrypt, variable PASSWORD_HASH). Un client VPN a été créé, générant une configuration et un QR code d'import.

J'ai déployé un serveur VPN WireGuard (via wg-easy, interface d'administration sur le port 51821) pour permettre un accès distant sécurisé. Un VPN crée un tunnel chiffré : une fois connecté, l'appareil se comporte comme s'il était physiquement sur le réseau local du NAS et peut accéder à tous ses services. J'ai créé un client VPN (« Mon téléphone ») avec sa

configuration et son QR code. Sur la maquette (VM en NAT, réseau fermé), la connexion réelle depuis l'extérieur n'est pas possible : il s'agit d'une démonstration du déploiement, le test réel se fera lors du déploiement sur mon réseau personnel.

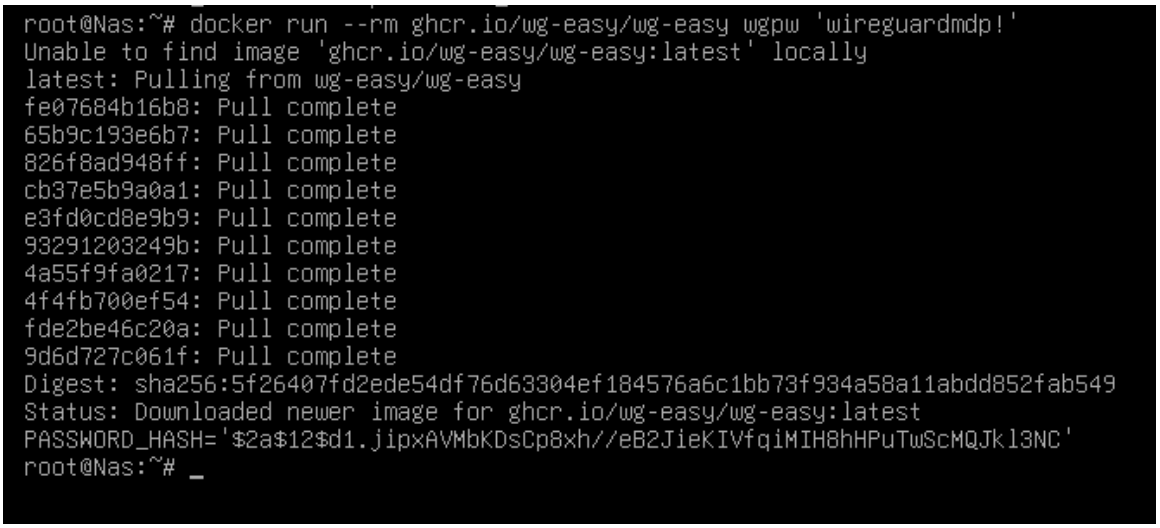


Figure 19 - Génération du mot de passe haché (bcrypt) pour l'interface WireGuard

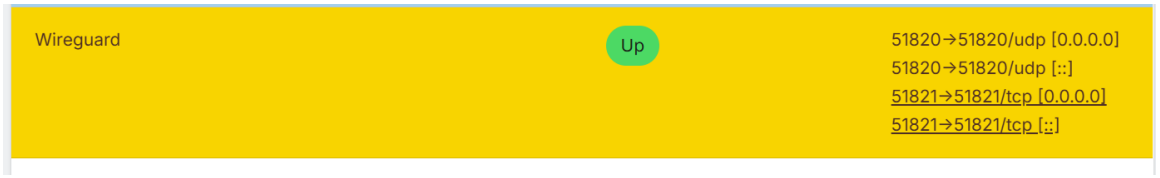


Figure 20 - Service WireGuard (wg-easy) en état « Up » — ports 51820/UDP et 51821/TCP

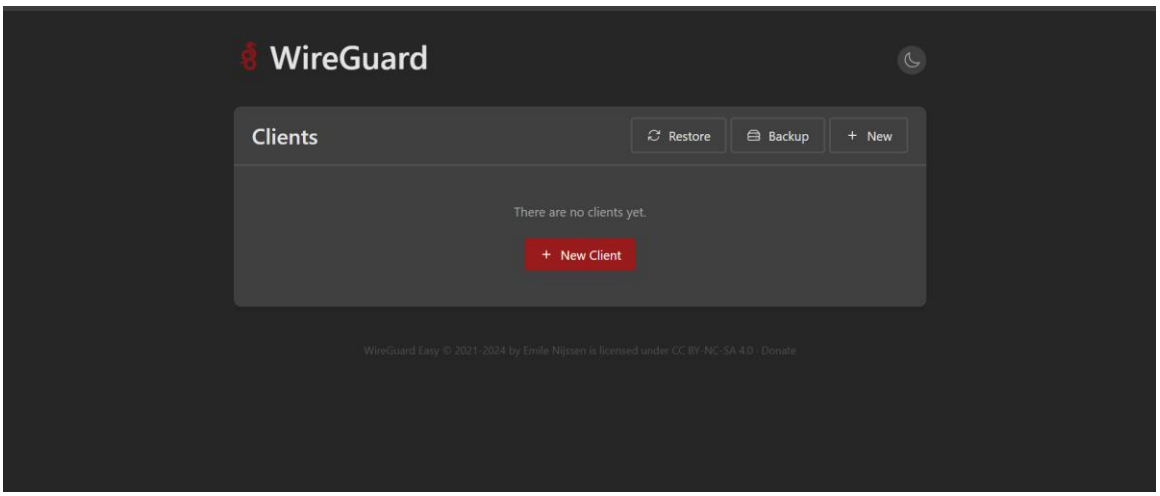


Figure 21 - Interface web de wg-easy (gestion des clients VPN)

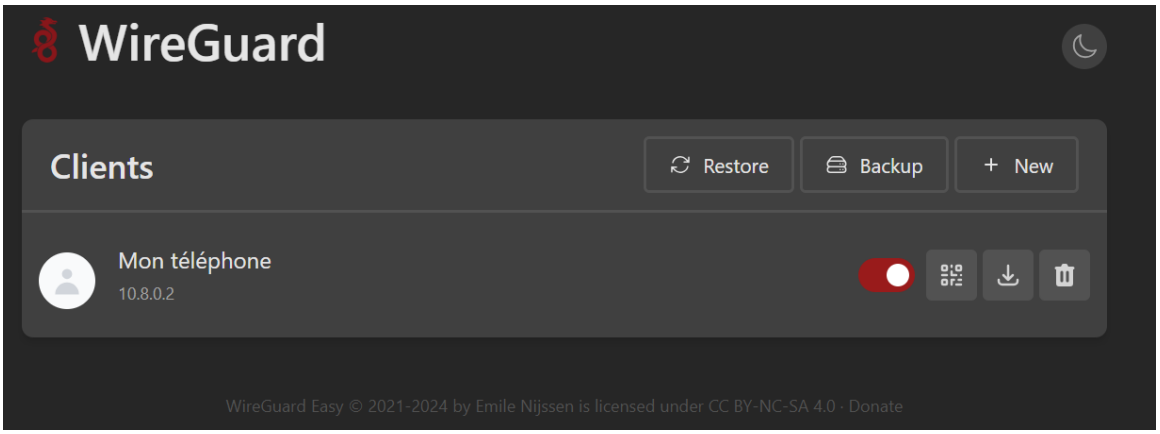


Figure 22 - Client VPN « Mon téléphone » créé (adresse 10.8.0.2)

7 ter. Hébergement du portfolio (serveur web Nginx)

Le portfolio (site statique HTML/CSS/JS, déjà publié sur GitHub Pages) a également été auto-hébergé sur le NAS via un conteneur Nginx (image nginx:alpine). Les fichiers du site sont stockés dans un dossier partagé et montés dans le conteneur en lecture seule (:ro). Le site est accessible sur le port 8082 : http://192.168.108.128:8082.

J'ai auto-hébergé mon portfolio (site statique, déjà publié sur GitHub Pages) sur le NAS, via un conteneur Nginx (port 8082), les fichiers du site étant montés en lecture seule. Cela illustre la différence entre un hébergement externe (GitHub Pages, public et clé en main) et l'auto-hébergement, où je maîtrise moi-même l'infrastructure et les données. J'ai rencontré un incident lié à la sensibilité à la casse sous Linux (le dossier « Portfolio » avec une majuscule étant différent de « portfolio »), qui provoquait une erreur 403 ; je l'ai corrigé en ajustant le chemin exact du dossier.

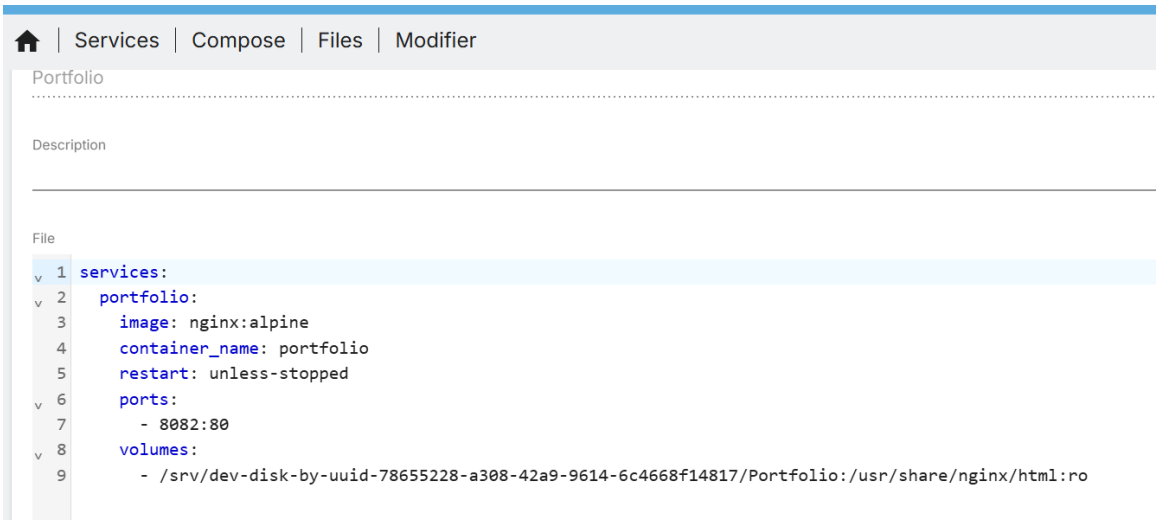


Figure 23 - Fichier Compose du portfolio (serveur Nginx, volume monté en lecture seule)

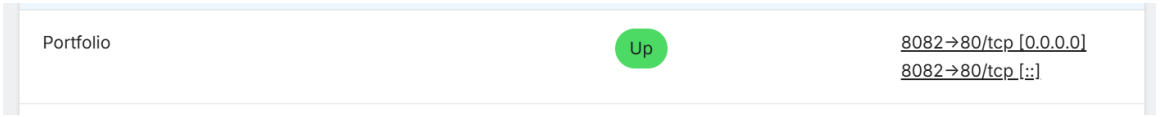


Figure 24 - Conteneur portfolio en état « Up » (port 8082)

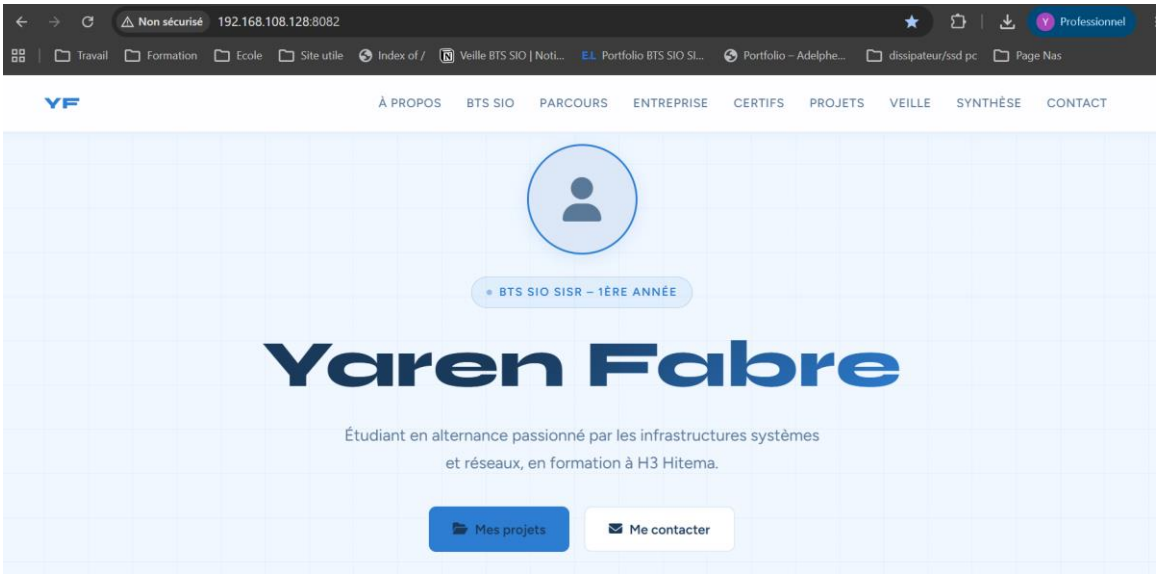


Figure 25 - Portfolio affiché depuis le NAS (http://192.168.108.128:8082)

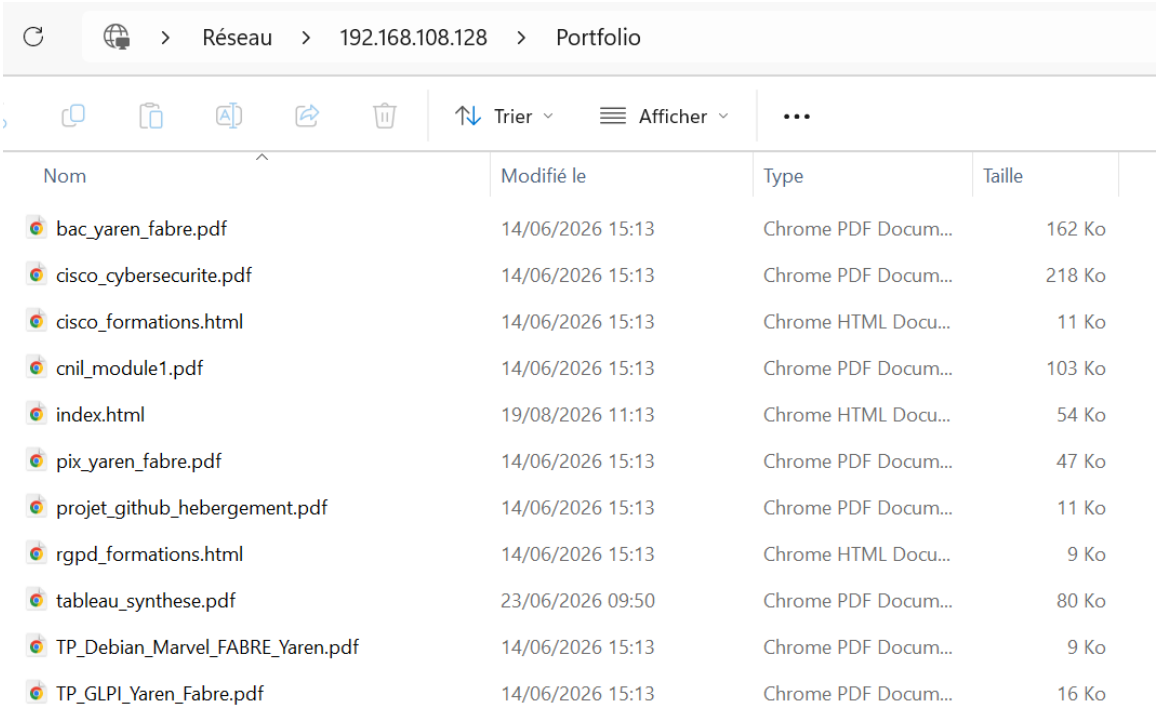
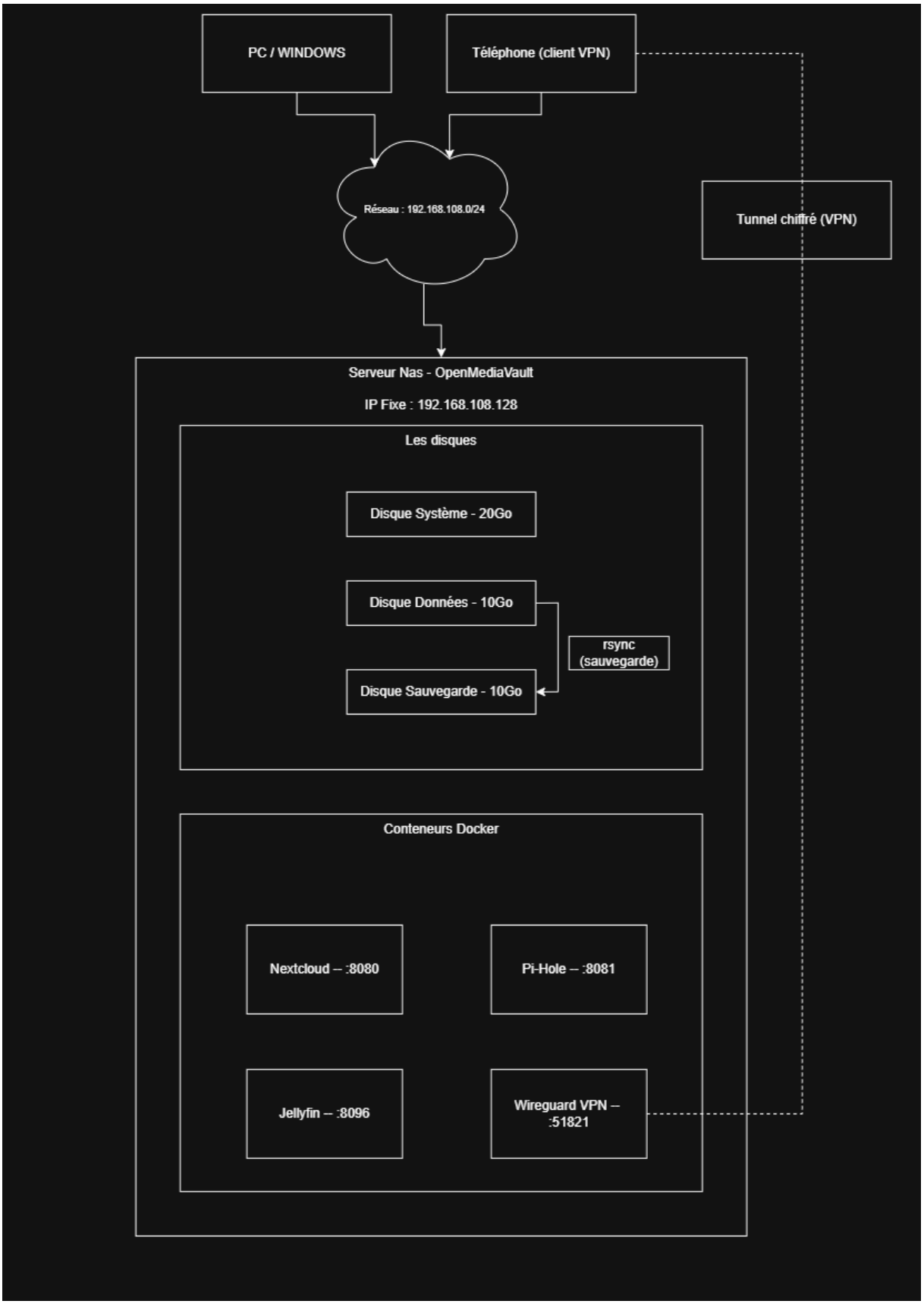


Figure 26 - Fichiers du portfolio stockés sur le NAS

- **8. Incidents rencontrés et résolus ☆ (ta meilleure section)**

Incident	Symptôme	Diagnostic	Solution
Adresse APIPA	169.254.x.x	carte réseau non connectée	reconnecter l'adaptateur VMware
MAJ interrompue	interface KO, réseau perdu	paquets à moitié installés	restauration snapshot
Horloge désynchro	signatures APT refusées, login KO	date de la VM dans le passé	timedatectl set-ntp true
Manque de ressources	rcu stalls, OOM	VM étranglée	fermer autres VM + reboot
Conflit de port	Pi-hole ne démarre pas	port 53 déjà pris (docker logs)	retirer le mapping DNS
Mot de passe conteneur	login Pi-hole refusé	mdp non pris par le compose	docker exec … setpassword

- Ce projet m'a confronté à plusieurs incidents que j'ai dû diagnostiquer et résoudre ; le tableau ci-dessous les récapitule. Trois exemples marquants : (1) après une mise à jour interrompue, la configuration réseau était corrompue et la VM n'avait plus d'adresse IP — l'interruption avait laissé des paquets à moitié installés, et je suis revenu à un snapshot sain. (2) L'installation de Docker échouait avec une erreur de signature APT (« repository is not signed ») : l'horloge de la VM était désynchronisée (revenue dans le passé après une restauration de snapshot), ce que j'ai corrigé en réactivant la synchronisation de l'heure (timedatectl set-ntp true). (3) Le conteneur Pi-hole refusait de démarrer à cause d'un conflit de port (le port 53 du DNS était déjà utilisé) : je l'ai identifié grâce aux journaux Docker (docker logs) et retiré le port en conflit. Ces incidents m'ont appris l'importance des snapshots et de la synchronisation de l'heure.
- **9. Schéma d'architecture**



- Le schéma ci-dessous représente l'architecture de la maquette : le serveur NAS sous OpenMediaVault (adresse IP fixe 192.168.108.128) avec ses trois disques (système,

données, sauvegarde) et ses services conteneurisés, relié au réseau local auquel se connectent le PC et le téléphone. Le téléphone accède au NAS via le tunnel chiffré WireGuard, et la sauvegarde est assurée par rsync entre le disque de données et le disque de sauvegarde.

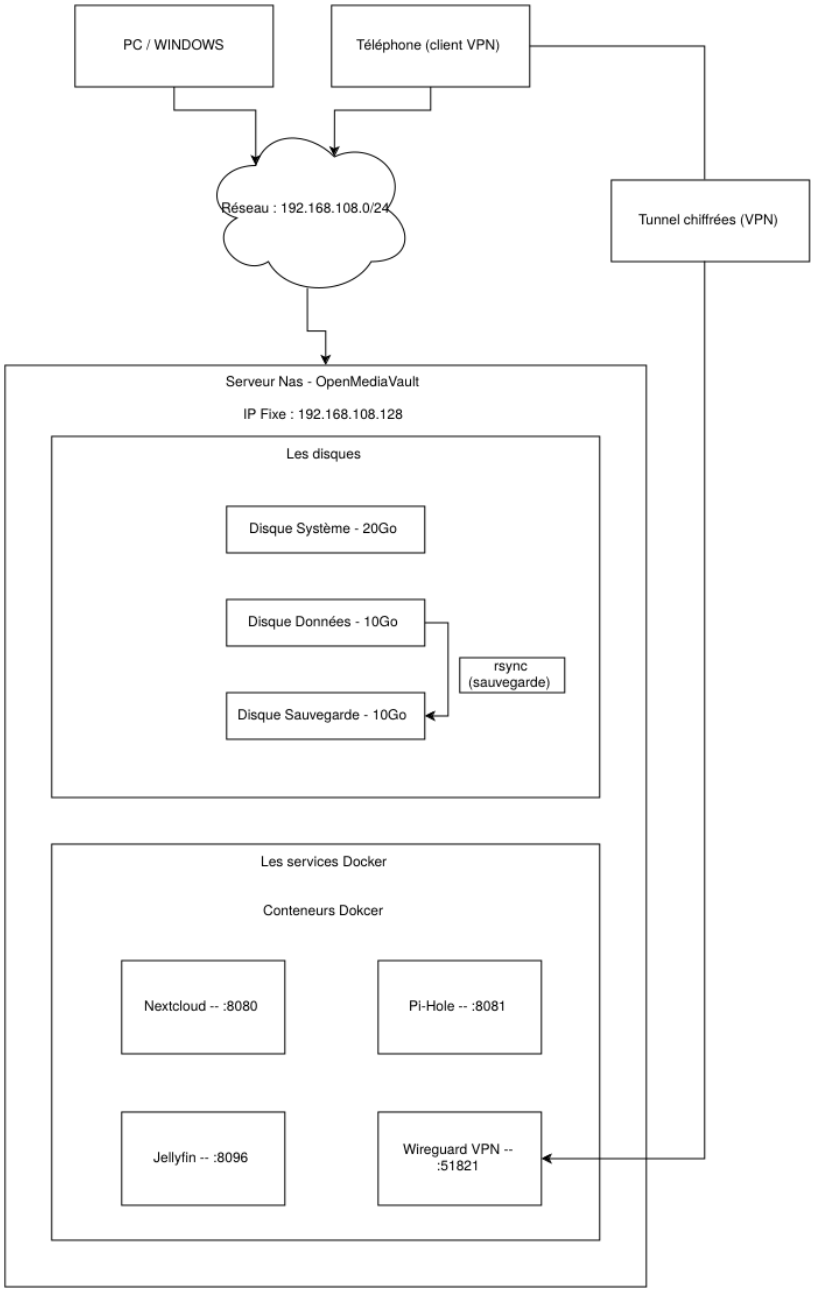


Figure 27 - Schéma d'architecture de la maquette NAS

- **10. Limites et perspectives**
- VM = labo d'apprentissage. Déploiement futur sur portable : install USB, disques physiques, réseau réel (Pi-hole filtrant, VPN WireGuard).
- Cette maquette constitue un environnement d'apprentissage et de test. J'ai également commencé à renforcer la sécurité : mise en place d'un pare-feu (politique

« tout bloquer sauf les ports utiles »), qui m'a fait comprendre la distinction entre trafic entrant et sortant, et déploiement d'un reverse proxy (Nginx Proxy Manager) pour centraliser l'accès et ajouter le HTTPS — partie encore en cours de configuration. Le déploiement réel se fera ensuite sur un ordinateur portable dédié, laissé allumé en permanence : installation via clé USB, disques physiques et réseau réel (Pi-hole en filtrage effectif, accès distant VPN pleinement fonctionnel avec DDNS et HTTPS). Cette démarche en deux phases (maquette puis production) est volontaire et méthodique.

- **11. Compétences BTS mobilisées**
- Administration système, stockage/partages, sauvegarde, réseau (IP, DNS, NTP), conteneurisation, diagnostic d'incidents, documentation.
- Les compétences mobilisées correspondent au référentiel du BTS SIO option SISR : installation et administration d'un système (OpenMediaVault / Debian), gestion du stockage et des partages, sauvegarde et restauration, configuration réseau (adressage IP statique, DNS, NTP), conteneurisation avec Docker, sécurisation (pare-feu, VPN, HTTPS) et surtout diagnostic et résolution d'incidents, ainsi que la documentation du projet.